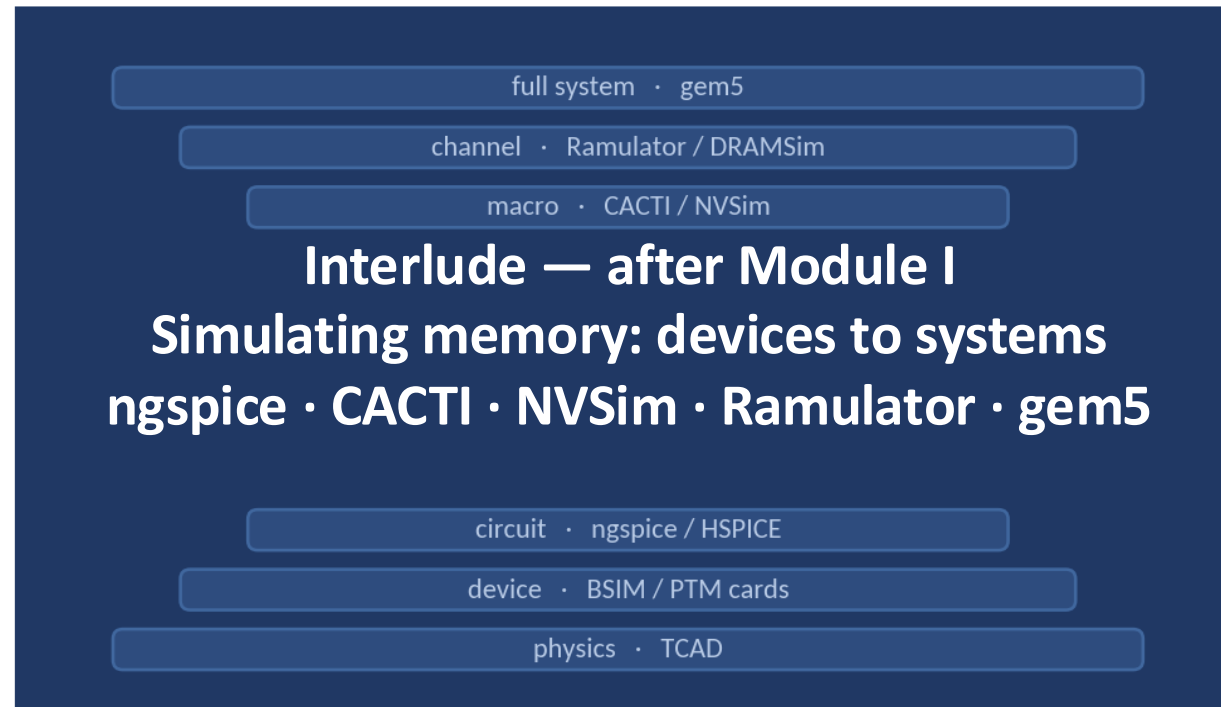


ECE2.414

Advanced Memory Circuits and Systems



Priyesh Shukla
IIIT Hyderabad · Monsoon 2026

Interlude outline

The ladder

why five tools and not one, and how the errors compound

ngspice

what SPICE really does, a 6T read, and an SNM sweep

CACTI · NVSim

the array as an optimisation problem, not a simulation

Ramulator · DRAMSim

the channel as a legal-move engine

gem5 (+ traces)

the whole machine, and what a trace throws away

One hands-on example per tool, all five on the same 2 MB L2 — and a reading list at the end that is meant to be used.

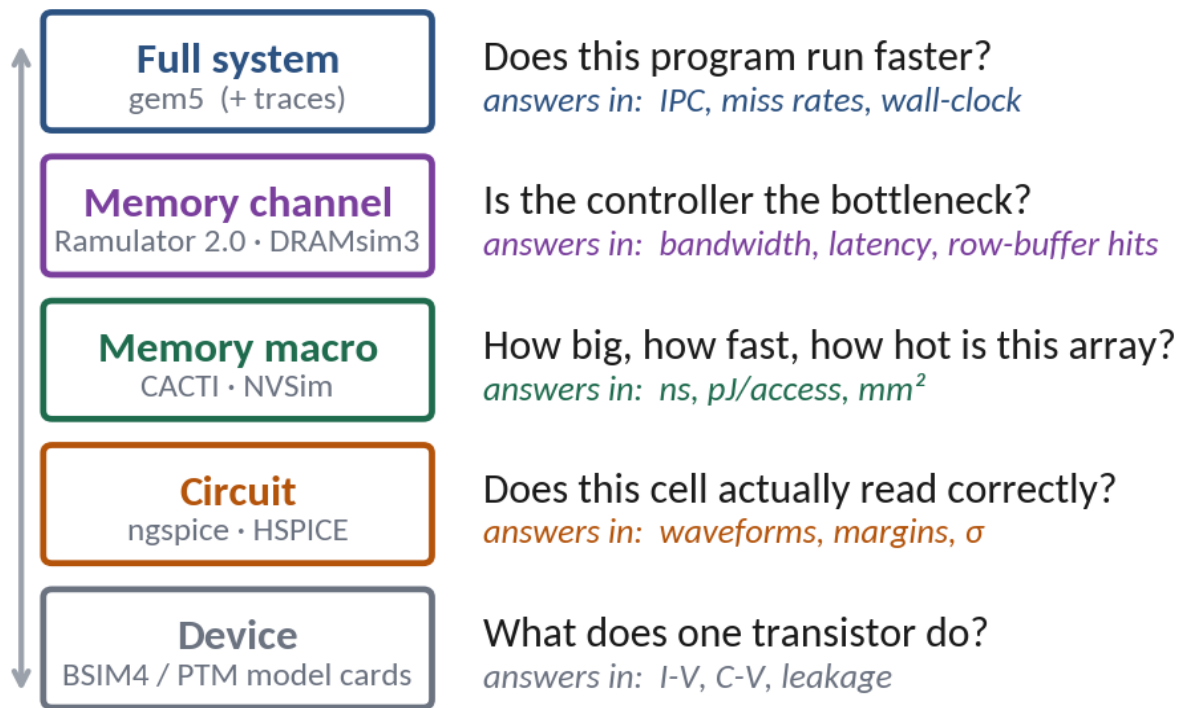
1

The ladder

Five tools, because there is no single tool that can span ten orders of magnitude.

One design question, five different answers

Each level treats the level below it as a fixed number — which is where the trouble starts



**one design question,
five different answers**

Each level takes the level below as a fixed number.

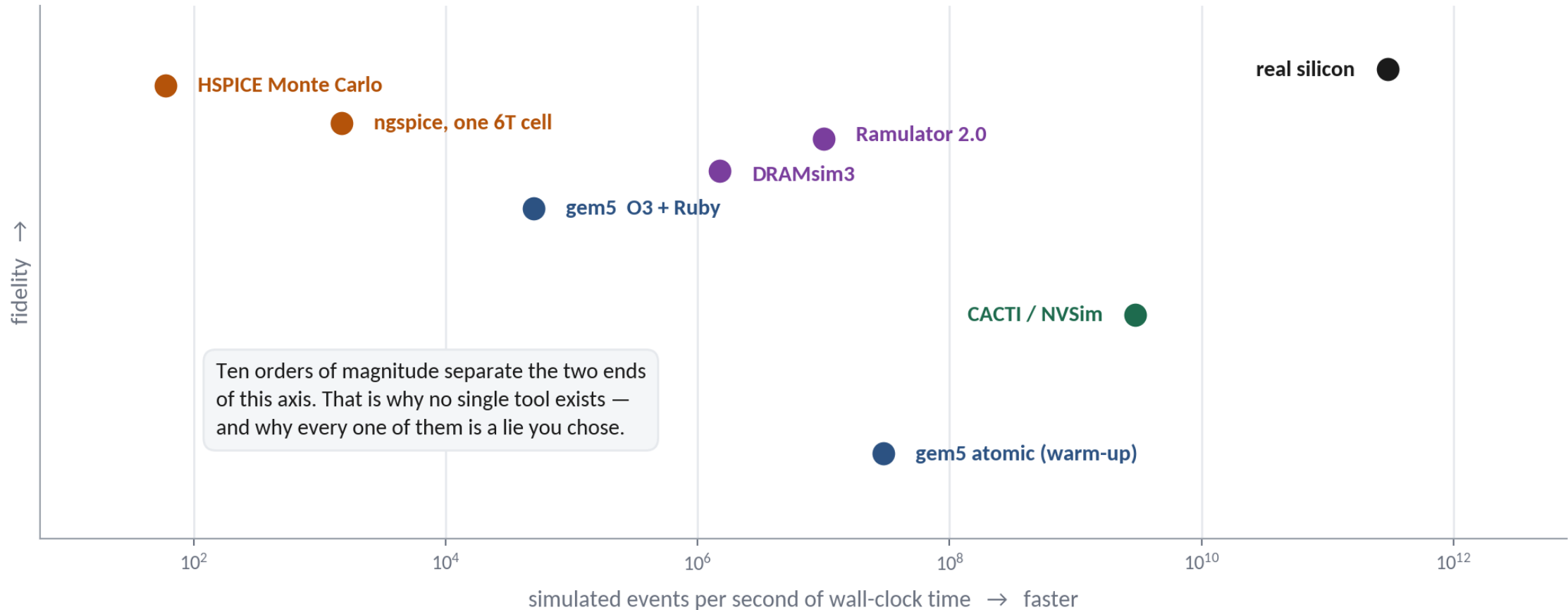
ngspice hands CACTI a delay. CACTI hands Ramulator a latency. Ramulator hands gem5 a bandwidth curve.

An error at the bottom propagates all the way up — silently.

Choosing a level is choosing which physics to stop modelling. Do that on purpose, not by accident.

The trade you are actually making

Speed and fidelity are not independent knobs — they are the same knob



The question we will carry through today

“Should the 2 MB L2 in our accelerator be SRAM, or STT-MRAM?”

ngspice

What is the read margin of the bitcell, at 0.7 V and 85 °C?

hours

CACTI

How fast, how big and how leaky is a 2 MB SRAM L2 at 45 nm?

seconds

NVSim

And the same 2 MB as STT-MRAM, with your device numbers?

seconds

Ramulator

Does the changed miss traffic still fit in one DDR4 channel?

minutes

gem5

Does the program you actually care about run faster?

hours

Not one of these five questions can be answered by any of the other four. That is the entire justification for owning five tools.

And notice: the answer to each one is a line in the next one's input file.

2

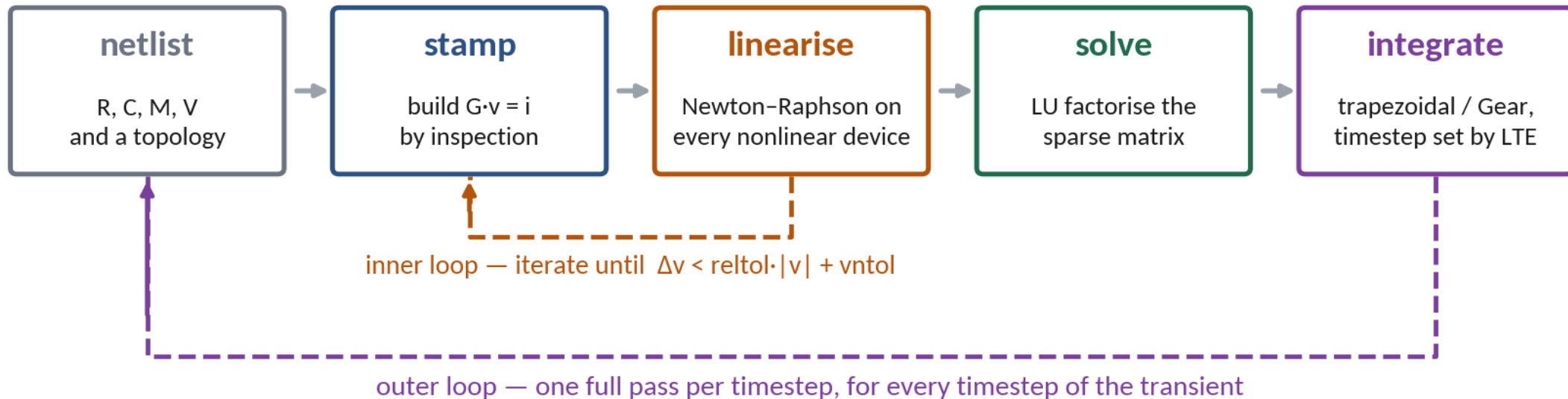
ngspice

The only tool today that solves physics. Everything above it solves arithmetic.

What SPICE is actually doing

Every error message you will ever see maps onto one of five boxes

Everything you will fight with in ngspice lives in one of these five boxes.



"convergence failure" = box 3 gave up · "timestep too small" = box 5 gave up · "it ran fine but the answer is wrong" = box 1 was wrong

SPICE does not "simulate a circuit". It repeatedly solves a linear system that approximates your circuit, and hopes the approximation converges.

ngspice, HSPICE, Spectre — what actually differs

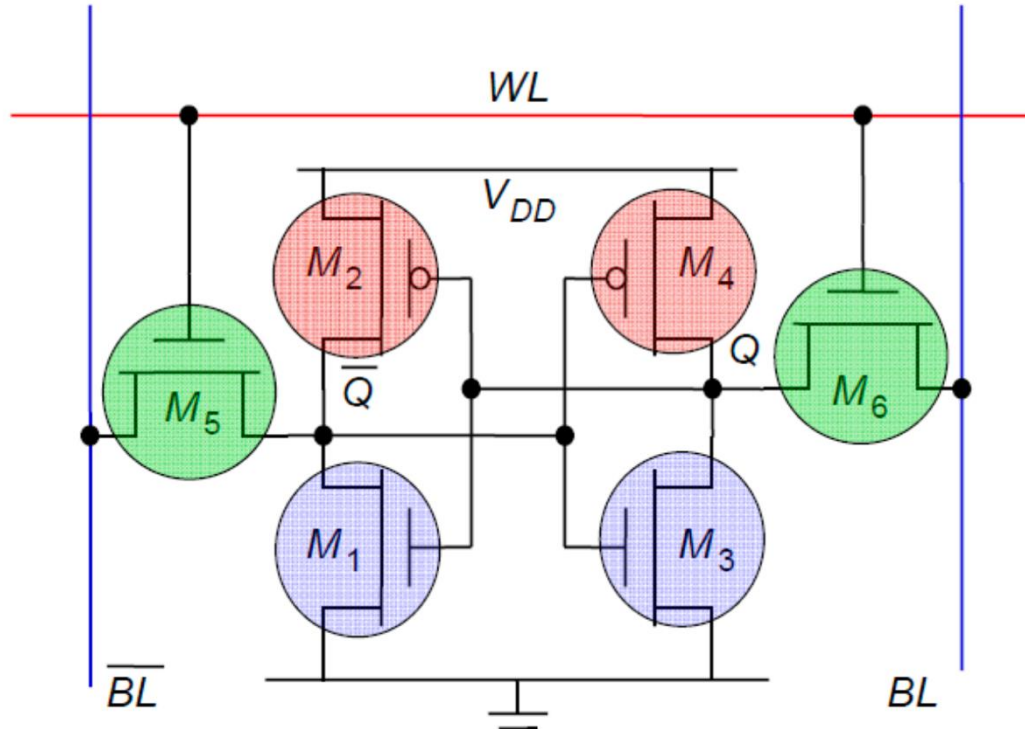
The netlists are 90 % portable. The last 10 % is where your weekend goes

	ngspice	HSPICE	Spectre
engine	SPICE3f5 + KLU, GPL	proprietary, heavily tuned	proprietary, RF-oriented
cost	free, apt-installable	site licence	site licence
models	BSIM3/4, BSIM-CMG, HICUM	everything, first	everything, first
control	.control block, its own shell	.alter / .measure sweeps	Spectre + SKILL
Monte Carlo	hand-rolled in .control loops	native .variation / .mc	native, plus corners
convergence	you will tune .options	usually just works	usually just works
use it for	coursework, papers, teaching	signoff, real silicon	RF and analogue signoff

We use ngspice because you can install it in ninety seconds and because your results are reproducible by anyone who reads your paper. Neither is true of the other two.

Hands-on case 1: read a 6T cell and measure the margin

The number every tool above this one will simply assume



What ngspice gives you here that no tool above it can:

- the actual $\Delta V(t)$ on the bitline pair, in volts
- the read current through the access device
- whether Q flips while you are reading it
- all of that at 85 °C, at 0.7 V, with V_T skew

The entire lab is this schematic. Twelve lines of netlist.

The netlist, and what to hand in

```
* AMCAS Lab - 6T read, bitline discharge and margin
.include 45nm_bulk.txt          $ PTM BSIM4 card
.param VDD=1.1 VBL=1.1
Vdd    vdd 0 'VDD'
Vwl    wl  0 PWL(0 0 1n 0 1.05n 'VDD')
* --- cross-coupled pair, Q=0 QB=1 ---
MP1 qb q vdd vdd pmos W=0.15u L=0.045u
MN1 qb q 0 0 nmos W=0.20u L=0.045u
MP2 q qb vdd vdd pmos W=0.15u L=0.045u
MN2 q qb 0 0 nmos W=0.20u L=0.045u
* --- access devices + bitlines ---
MA1 bl wl q 0 nmos W=0.16u L=0.045u
MA2 blb wl qb 0 nmos W=0.16u L=0.045u
Cbl bl 0 180f IC='VBL'
Cblb blb 0 180f IC='VBL'
.ic v(q)=0 v(qb)='VDD'
.control
    tran 5p 4n uic
    meas tran dv FIND v(blb)-v(bl) AT=2.0n
    meas tran qmax MAX v(q) FROM=1n TO=4n
    wrdata read.csv v(bl) v(blb) v(q) v(qb)
.endc
.end
```

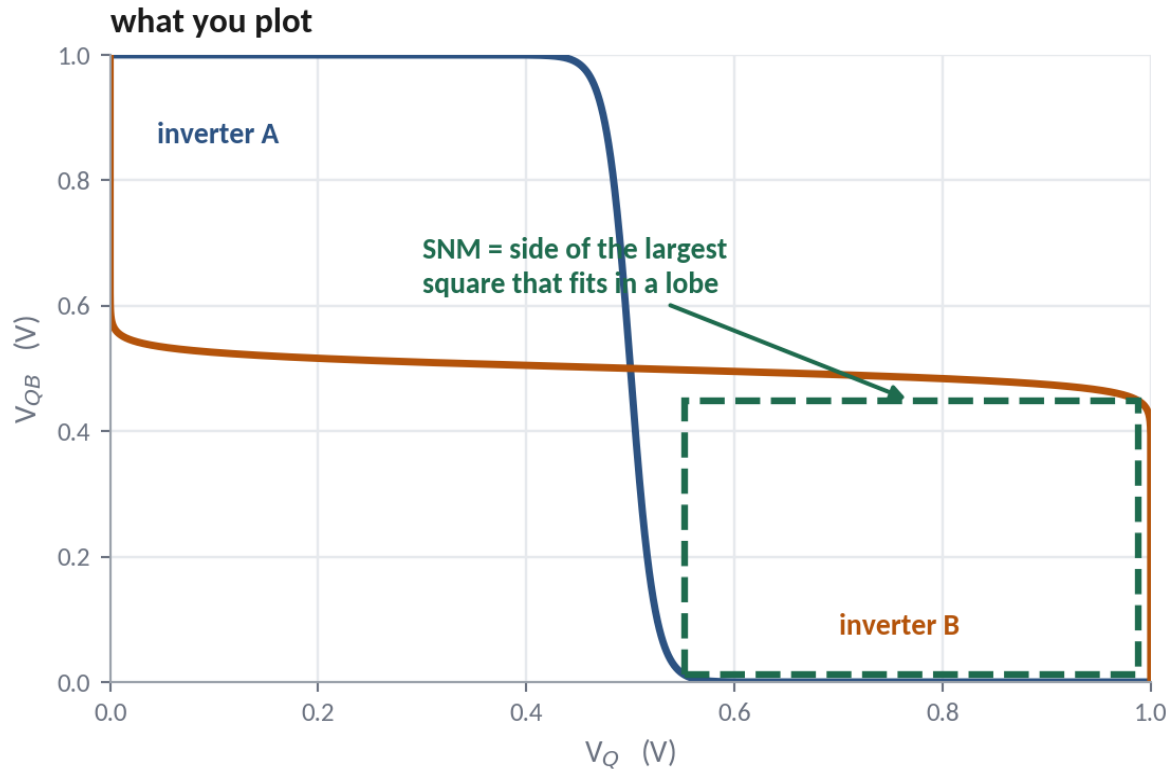
- 1 Report $\Delta V(\text{BL}, \text{BLB})$ at 2 ns. Compare it with the 69 mV you were handed in Lecture 1.
- 2 Plot $v(q)$. Does it move during the read? Grow MA1/MA2 to 0.24 μm and re-run — now what?
- 3 Sweep V_{DD} from 1.1 V down to 0.6 V in 50 mV steps with a .control loop. Plot $\Delta V(V_{\text{DD}})$ and find where it falls below your Lecture 5 offset of 25 mV.
- 4 Repeat task 1 at .temp 85. One paragraph: which of ΔV and the sense-amp offset moved more, and which one sets the failure?

The cell ratio here is deliberately marginal. Task 2 is meant to fail.

Everything CACTI tells you in the next section is a curve fit to results like this one.

The same tool, a different sweep: static noise margin

Two DC sweeps and a rotation — the whole of Lecture 7, prefigured



how ngspice produces it

- 1 Break the loop.
Two open inverters, not one latched cell.
- 2 Drive A's input, sweep $0 \rightarrow V_{DD}$.
dc vin 0 1.0 1m
- 3 Repeat for B.
Only reuse A by symmetry if you sized them identically.
- 4 Rotate 45° , take max of the difference.
The square-fitting happens in post-processing, not in SPICE.

A latched cell is bistable — it has no VTC to sweep.
Every SNM extraction begins by cutting the feedback.

```
.control
dc vin 0 1.1 1m          $ inverter A, loop cut
wdata vtcA.csv v(vout)
alter @Vin[dc] = 0        $ then repeat for B
.endc
```

SNM is not a SPICE output. It is a geometric construction you perform on two SPICE outputs — which is why every group's number differs slightly, and why you must state your method.

Six ways ngspice will quietly lie to you

Each of these has cost a student in this course a week. In some cases, a chapter

Level 1 models

have no subthreshold conduction. Your retention time becomes infinite and your leakage becomes zero — silently. Always use a BSIM4 card.

A symmetric netlist

has exactly zero offset and will resolve a 1 μ V input perfectly. That is the simulator's symmetry, not physics. Impose mismatch, or run Monte Carlo.

Default tolerances

reitol = 1e-3 and abstol = 1e-12 are fine for logic and useless for leakage. Set abstol = 1e-18 before you believe a femtoampere.

No parasitics

unless you put them there. A netlist without extracted R and C is a schematic-level answer wearing a layout-level costume.

“Convergence achieved”

means the solver stopped, not that the answer is right. Halve the tolerances and re-run: if the answer moves, it was never converged.

One corner

is not a result. TT at 27 °C at nominal V_{DD} is the one operating point your chip will never actually be in.

None of these produce an error message. All of them produce a plausible number.

3

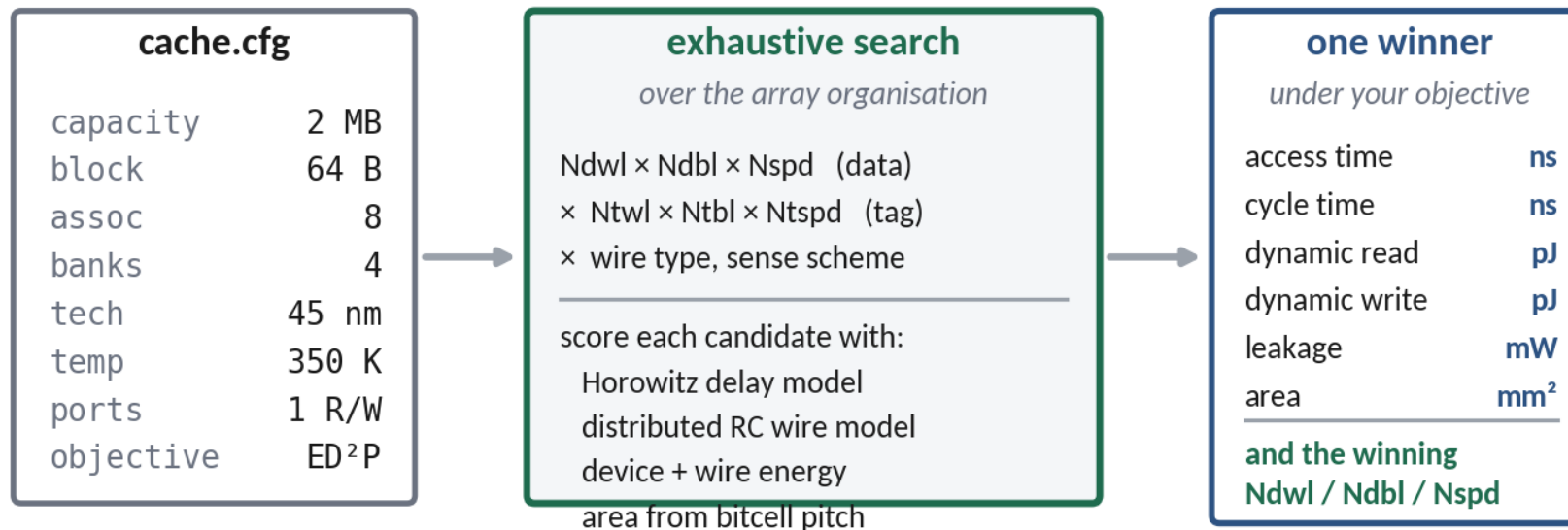
CACTI · NVSim

Stop simulating the array. Start searching it.

CACTI does not simulate. It optimises.

Analytical delay and energy models, wrapped in an exhaustive search over organisations

CACTI does not simulate a single transistor. It solves an optimisation problem.



The point is not the number.

It is that CACTI tries tens of thousands of array organisations before it reports one — the sweep you did by hand in Lecture 4, done exhaustively.

Change the objective and the winning organisation changes. That is the experiment.

Which is exactly why it runs in 200 ms — and why it is 6–10 % off a SPICE answer, on a good day.

Lineage worth knowing: Wilton & Jouppi built it in 1996 for caches; CACTI 6 added interconnect and NUCA; CACTI 7 added off-chip I/O, HBM and buffered DIMMs.

Hands-on case 2: size the 2 MB L2 with CACTI

Runtime and sweep

```
# cache.cfg (CACTI 7)
-size (bytes) 2097152
-block size (bytes) 64
-associativity 8
-read-write port 1
-UCA bank count 4
-technology (u) 0.045
-operating temperature (K) 350
-cache type "cache"
-Data array cell type - "itrs-hp"
-Data array peripheral type - "itrs-hp"
-Optimize ED or ED^2 (ED, ED^2, NONE): "ED^2"
-design objective (weight delay, dynamic power,
    leakage power, cycle time, area) 0:0:0:100:0

$ ./cacti -infile cache.cfg
```

What comes back: access time, cycle time, dynamic read/write energy, leakage per bank, area and area efficiency — and the winning N_{dwl} / N_{dbl} / N_{spd} it chose to get there.

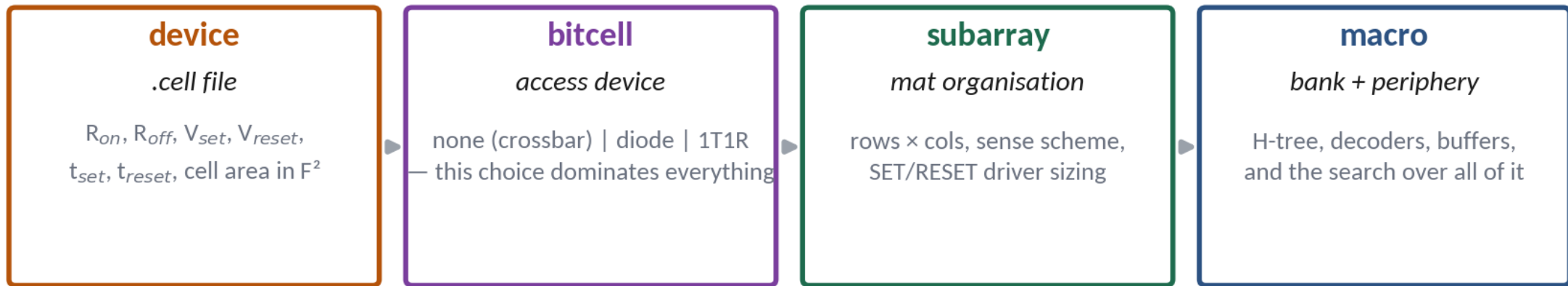
Task 3 is the one that matters. There is no such thing as “the optimal cache” — only the optimum of the objective you wrote down.

- 1 Run it. Record access time, dynamic read energy, leakage and area. This is your SRAM baseline for the rest of the assignment.
- 2 Sweep capacity 256 kB → 16 MB, everything else fixed. Plot access time against log capacity. Is Amrutur & Horowitz's “one gate delay per doubling” visible?
- 3 Now change only the objective — pure delay, then pure area, then ED²P. Report the three winning N_{dwl}/N_{dbl} triples. The organisation changes; explain why.
- 4 Take CACTI's bitline delay for your winner and hand-check it against $0.38 \cdot R \cdot C \cdot L^2$ from Lecture 4. Report the discrepancy and name one thing CACTI models that you did not.

NVSim: the same machine with the bitcell pulled out

If you understand CACTI you already understand NVSim — the new part is one text file

NVSim is CACTI with the bitcell swapped out — and the bitcell is a text file you write.



This is the whole reason NVSim exists: the array, periphery and search are unchanged from CACTI. Only the cell model is new.

Which also means: if your device numbers are guesses, so is everything downstream of them.

Dong, Xu, Xie & Jouppi validated it against industrial PCM, STT-RAM and ReRAM prototypes. Your results are only as good as the .cell file you feed it.

Hands-on case 3: the same 2 MB, as STT-MRAM

Change the bitcell, keep everything else identical

```
-MemCellType: MRAM
-CellArea (F^2): 54
-CellAspectRatio: 2.0
-ResistanceOn (ohm): 3000
-ResistanceOff (ohm): 6000
-ReadMode: current
-ReadCurrent (uA): 40
-ReadPower (uW): 145
-ResetMode: current
-ResetCurrent (uA): 200
-ResetPulse (ns): 10
-SetMode: current
-SetCurrent (uA): 200
-SetPulse (ns): 10
-AccessType: CMOS
-AccessCMOSWidth (F): 6

$ ./nvsim ./config/STT_cache.cfg
```

sample.cell — sixteen lines, and every architectural conclusion you draw rests on them

Task 4: the write current sizes the access transistor, the access transistor sets the cell area, and the cell area sets the density advantage you bought the technology for.

- 1 Run it at 2 MB with the shipped STT-RAM cell. Tabulate read latency, write latency, read energy, write energy, leakage and area against your CACTI SRAM baseline.
- 2 Two of those six will be dramatically worse and two dramatically better. Say which, and why, in device terms — not “because the tool said so”.
- 3 The TMR here is 2:1 (3 k Ω / 6 k Ω). Re-run at 4 k Ω / 12 k Ω . Which output moved most? Now you know what a device paper's TMR headline actually buys.
- 4 Halve ResetCurrent to 100 μ A and re-run. Note what happened to area, and explain the mechanism — this is the single most important coupling in NVM design.

What CACTI and NVSim do not know about your design

Read the output, then read this list, then decide how much of the output to believe

Process variation

Everything is nominal. There is no σ , no yield, no repair. Your 2 MB array has 16 million cells and all of them are typical.

Your actual layout

Area comes from a cell-pitch model, not a floorplan. Routing congestion, power-grid area and DFT overhead are simply absent.

Real wire stacks

One idealised metal stack per technology node. No via resistance ladders, no local-vs-global routing decisions.

Anything above 22 nm-ish

The device models are ITRS-era planar. FinFET and GAA are approximated, if present at all. Beware confident deep-node numbers.

Thermal coupling

Temperature is an input you set, not a state that evolves. A leaky array does not heat itself up and get leakier.

Your control logic

Tag comparators, replacement state, coherence directories — not modelled, and at high associativity they are not small.

Use these tools for ratios and trends, which survive — not for absolute numbers.

4

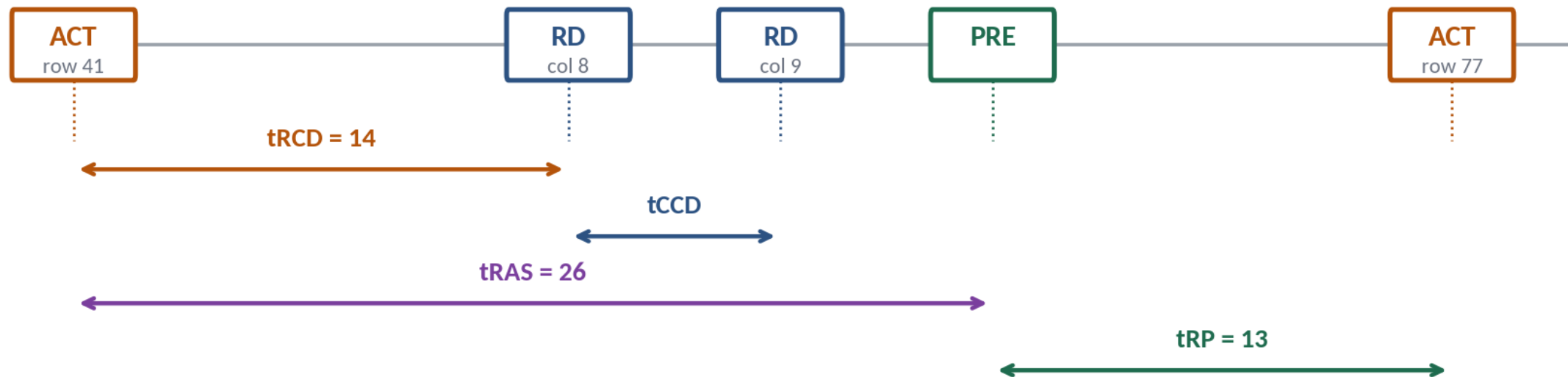
Ramulator · DRAMSim

No charge. No capacitor. No sense amplifier. Only legal moves and a clock.

A DRAM simulator is a legal-move engine

Everything becomes a table of integers and a scheduler

There is no charge, no capacitor, no sense amplifier anywhere in a DRAM simulator. *Only legal-move arithmetic.*
command bus



A DRAM simulator is exactly this: a table of constraints, a state per bank, and a scheduler that picks the next legal command.

The physics was compiled away into tRCD, tRP, tRAS and tCCD. That compilation is what buys you a factor of a billion in speed — and it is exactly what you lose.

Ramulator 2.0 and DRAMsim3 — pick one and know why

Both are cycle-accurate, both are free, and they will not give you the same answer

	Ramulator 2.0	DRAMsim3
written in	C++20, plugin architecture	C++11, compact and stable
standards	DDR3/4/5, LPDDR5, GDDR6, HBM(2)/3	DDR3/4, LPDDR3/4, GDDR5/6, HBM
built for	adding new standards and RowHammer mitigations as plugins	validated timing and thermal modelling
thermal	no	yes — the reason it exists
standalone	yes, trace-driven	yes, trace-driven
as a library	gem5, zsim	gem5, and a simple C++ callback API
lineage	Kim, Yang & Mutlu 2016 → Luo et al. 2024	DRAMSim2 2011 → DRAMsim3 2020

If you are studying a memory controller policy, reach for Ramulator. If you need a number you will depend on thermals or timing fidelity, reach for DRAMsim3. If they disagree by more than 10 %, one of your configs is wrong.

Hands-on case 4: put your L2 miss stream on a DDR4 channel

The traffic your cache decision actually creates

```
# ddr4.yaml (Ramulator 2.0)
Frontend:
  impl: SimpleO3
  traces: [ l2miss.trace ]
  clock_ratio: 8
MemorySystem:
  impl: GenericDRAM
  clock_ratio: 3
  DRAM:
    impl: DDR4
    org: { preset: DDR4_8Gb_x8, channel: 1, rank: 2 }
    timing: { preset: DDR4_3200AA }
  Controller:
    impl: Generic
    Scheduler: { impl: FRFCFS }
    RefreshManager: { impl: AllBank }
    plugins: [ { ControllerPlugin: { impl: TraceRecorder } } ]

# l2miss.trace - one access per line
0x7f2a4c00 R
0x7f2a4c40 R
0x7f2a5000 W

$ ./ramulator2 -f ddr4.yaml
```

- 1 Run it. Report memory_cycles, total read/write latency and the row-buffer hit rate from the stats dump.
- 2 Switch Scheduler from FRFCFS to FCFS. Row-buffer hit rate will collapse. Quantify the latency cost, and explain it in terms of tRCD and tRP.
- 3 Change the address mapping so the row bits sit below the bank bits. Re-run. You have just destroyed bank-level parallelism — by how much?
- 4 Now double the channel count. If latency barely improves, say which of the four DRAM timing parameters is binding, and how you would know.

Generate the trace yourself: the CommMonitor on your gem5 L2's memory-side port dumps exactly this format.

5

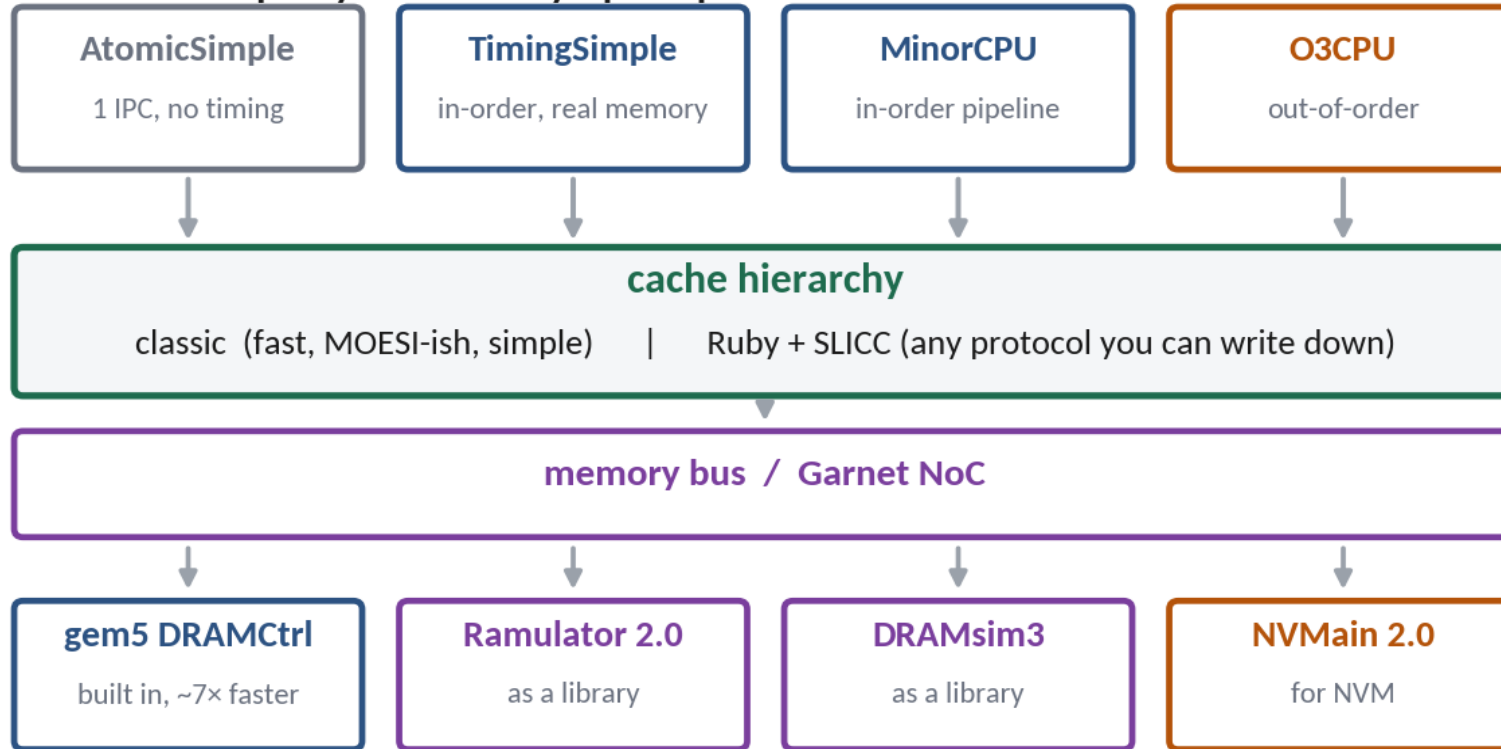
gem5 (+ traces)

The only tool today that can answer “does the program run faster?”

gem5, for people who care about memory

Four decisions, and only the last two are about memory at all

CPU model — pick your accuracy/speed point



Two modes

SE — syscalls emulated. Starts in seconds. No OS, no page tables you trust, no I/O.

FS — boots a real Linux. Honest TLB and page-cache behaviour. Costs you twenty minutes of boot.

For memory research the difference is not cosmetic: TLB misses and page allocation change your address stream.

Traces: what they buy, and what they quietly throw away

A trace is a program with the feedback removed — sometimes that is fine, often it is not

What you gain

Reproducibility, speed, and the ability to drive Ramulator or DRAMsim3 without running gem5 at all. A trace is a file; a workload is a build system.

What you lose

Every dependence between memory latency and future behaviour. A slower memory no longer delays the next request — the trace has already decided when it arrives.

Why that matters

Any study of prefetching, MLP, scheduling or out-of-order tolerance is distorted by a fixed-arrival trace. The effect you are measuring is the one you removed.

The fix

gem5's elastic traces (Trace CPU) record the dependence graph, not just the addresses, and replay it with timing that responds to the memory. Slower than a flat trace, honest.

Where traces come from

gem5 CommMonitor on a cache's memory-side port, Pin/DynamoRIO instrumentation, or the published suites. Say which, always.

Warm-up

A cold 2 MB L2 needs tens of millions of accesses before its miss rate is meaningful. Fast-forward atomically, checkpoint, then switch CPUs.

Rule of thumb: a flat address trace is honest for bandwidth questions and dishonest for latency-sensitivity questions.

Hands-on case 5: run the whole thing

The only part where the answer is in cycles, not picojoules

```
# baseline: your CACTI SRAM L2
$ build/X86/gem5.opt configs/deprecated/example/se.py \
  --cpu-type=O3CPU --caches --l2cache \
  --l1d_size=32kB --l1i_size=32kB \
  --l2_size=2MB --l2_assoc=8 \
  --l2-hit-latency=7 \           # <- from CACTI, in cycles
  --mem-type=DDR4_2400_8x8 --mem-size=4GB \
  --cmd=./bfs --options="-g 18"

# then the NVSim STT-MRAM L2: slower reads, much slower
# writes, but you can afford 8 MB in the same area
$ ... --l2_size=8MB --l2-hit-latency=14

# read the answers out of m5out/stats.txt
$ grep -E "simSeconds|ipc|l2.*miss_rate" m5out/stats.txt
```

Note the two numbers carried in from earlier parts: the L2 hit latency comes from CACTI and NVSim, and the capacity trade comes from their area outputs. gem5 invents neither.

1

Run both configurations on two GAPBS kernels. Report IPC, L2 miss rate and simSeconds.

2

The MRAM configuration is 4× larger and roughly 2× slower per hit. For which kernel does that trade win, and what property of the kernel decides it?

3

Re-run the winner with --cpu-type=TimingSimple. The advantage will change size. Explain what out-of-order execution was doing to your result.

4

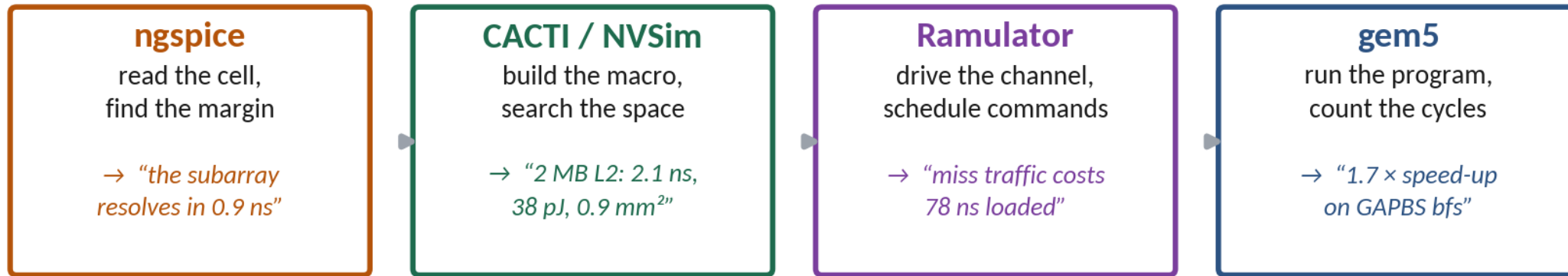
Two paragraphs, and this is the deliverable: answer the question from slide 6, and state the three assumptions in your chain you would attack first if you were the reviewer.

Task 4 carries the marks. Tasks 1 to 3 were the measurements it is made of.

The whole thing, on one slide

Every arrow is a file, and every file is a set of assumptions you are now responsible for

The output of each tool is an input parameter of the next. That is the whole toolchain.



± 5 %

± 10 %

± 10 %

± 20 %

Errors compound multiplicatively along this chain. Report the ratio between two configurations, not the absolute number — the ratio is what survives.

“STT-MRAM is 1.7× faster” is not a result. “STT-MRAM is 1.7× faster on bfs, given these device numbers, this cell file, this scheduler and an O3 core” is.

How wrong is each tool, honestly?

Approximate, and quotable only with the caveats — but you should know the order of magnitude

ngspice vs. silicon

a few percent, if your model card matches your process. If it does not, unbounded.

the model card is the experiment

CACTI vs. SPICE

roughly 6 % on access time, by its own validation. Worse at nodes it was never fitted to.

trends survive, absolutes do not

NVSim vs. prototypes

validated against published PCM, STT-RAM and ReRAM parts — for those parts.

your .cell file is the weak link

Ramulator / DRAMsim3

each validated against vendor models; they can still differ from each other by ~10 %.

run both, and explain the gap

gem5 vs. real hardware

commonly 10–30 % on absolute cycle counts, worse on unvalidated configs.

never quote absolute IPC

The professional habit these numbers should produce: state your tool, its version, its config and its validation basis, and report differences rather than absolutes. Every serious memory paper you will read this semester does exactly this — go and check.

Assignment — one question, five tools

Install

ngspice (apt / brew), CACTI 7 and NVSim (make), Ramulator 2.0 (CMake, C++20), gem5 (scons, expect 40 min). Docker images for all five are on the course page.

Model cards

PTM 45 nm bulk BSIM4 from ptm.asu.edu for part A. Everything else uses the shipped 45 nm configs so your five results stay comparable.

Deliverable

One PDF, six pages maximum: five tables, four plots, and the two paragraphs from part E task 4.

Marking

40 % on the part E argument, 40 % on the sweeps, 20 % on whether your numbers are reproducible from what you wrote down.

Working in pairs

Encouraged. Different pairs will get different answers, and comparing them in Lecture 9 is the point.

Before the next lecture

Read

The five references on the next slide — skim four, read Kim, Yang & Mutlu properly. It is five pages.

Do

Assignment 3 part A only, before Lecture 7. The rest follows the modules.

Install

All five tools this week, not the night before. gem5 in particular.

Think about

You now have a tool that reports an SNM number and a lecture coming that defines one. Which is prior? What would it mean for the tool and the definition to disagree?

Next lecture — back to Module II: the 6T SRAM cell. Read, write and hold; the conflict between read stability and writeability; and why the sizing ratios are not free parameters.

Five papers — one per tool

Each one is the paper you cite when a reviewer asks where your numbers came from

- 1** S. J. E. Wilton and N. P. Jouppi, “CACTI: An Enhanced Cache Access and Cycle Time Model,” *IEEE Journal of Solid-State Circuits*, vol. 31, no. 5, pp. 677–688, May 1996.
The analytical model under everything in section 3. For the modern tool, cite instead R. Balasubramonian et al., “CACTI 7,” ACM TACO, vol. 14, no. 2, art. 14, June 2017.
- 2** X. Dong, C. Xu, Y. Xie and N. P. Jouppi, “NVSim: A Circuit-Level Performance, Energy, and Area Model for Emerging Nonvolatile Memory,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 31, no. 7, pp. 994–1007, July 2012.
Read section IV, on the cell model — it is what your .cell file in part C is actually feeding.
- 3** Y. Kim, W. Yang and O. Mutlu, “Ramulator: A Fast and Extensible DRAM Simulator,” *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, Jan.–June 2016.
Five pages, and the clearest short account anywhere of how a DRAM standard becomes a state machine. Then H. Luo et al., “Ramulator 2.0,” IEEE CAL, vol. 23, no. 1, pp. 112–116, 2024, for the tool you will actually run.
- 4** S. Li, Z. Yang, D. Reddy, A. Srivastava and B. Jacob, “DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator,” *IEEE CAL*, vol. 19, no. 2, pp. 106–109, July–Dec. 2020.
- 5** N. Binkert et al., “The gem5 Simulator,” *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, May 2011 — and J. Lowe-Power et al., “The gem5 Simulator: Version 20.0+,” arXiv:2007.03152, 2020, which is the one that matches the code you will build.

Where to get everything

Bookmark this slide — you will be back for it in Modules III, IV and VI

ngspice	<code>ngspice.sourceforge.io</code>	manual is 700 pages; chapters 15–17 are the ones you need
device cards	<code>ptm.asu.edu</code> · <code>github.com/google/skywater-pdk</code>	PTM for teaching; SkyWater 130 nm for a real PDK with corners
CACTI 7	<code>github.com/HewlettPackard/cacti</code>	make, then read the shipped <code>cache.cfg</code> comments before anything else
NVSim	<code>github.com/SEAL-UCSB/NVSim</code>	see also DESTINY, for 3D and eDRAM arrays
Ramulator 2.0	<code>github.com/CMU-SAFARI/ramulator2</code>	the <code>.devcontainer</code> Dockerfile saves you the dependency fight
DRAMsim3	<code>github.com/umd-memsys/DRAMsim3</code>	<code>configs/</code> has validated presets for most DDR4 and HBM parts
gem5	<code>gem5.org</code> · <code>learning.gem5.org</code>	the learning site is a genuine tutorial, not marketing
workloads	<code>resources.gem5.org</code> · GAPBS, PARSEC, SPEC	<code>gem5-resources</code> ships prebuilt disk images and kernels
SAFARI lectures	<code>youtube.com/@onurmutlulectures</code>	full memory-systems courses, free, and the Ramulator tutorials

All five tools are free and all five run on a laptop. There has never been a better moment to be a memory researcher with no fab.